The following is a step-by-step explanation of how streaming typically works in a web application:

1. *Client request*—The client (web browser) sends a request to the server, indicating that it wants to receive a streaming response.
2. *Server processing*—The server receives the request and initiates processing the AI-generated response. Instead of waiting for the entire response to be generated, the server starts sending data chunks as soon as they are available.
3. *Chunked transfer encoding*—The server uses a technique called *chunked transfer encoding* to break down the response into smaller chunks. Each chunk is preceded by its size and followed by a newline character. The following is an example of chunked transfer encoding. Suppose the server is sending a response with the following content:

   ```
   Hello, world! This is an example of chunked transfer encoding.
   ```

   Using chunked transfer encoding, the server would break down the response into smaller chunks and send them as follows:

   ```
   1F\r\n
   Hello, world! This is an ex
   \r\n
   1D\r\n
   ample of chunked transfer encoding.
   \r\n
   0\r\n
   \r\n
   ```

   When the client receives this chunked response, it reads the size of each chunk, processes the data, and continues reading the next chunk until it encounters the chunk with size 0, indicating the end of the response.
4. *Incremental delivery*—As the server generates and processes the AI response, it sends the data chunks to the client incrementally. The client receives these chunks as they arrive, without waiting for the complete response.
5. *Client-side processing*—The client receives the data chunks and processes them as they arrive. It can update the user interface incrementally, displaying the partially generated content to the user.
6. *Completion*—When the server finishes generating the complete AI response, it sends a final chunk indicating the end of the stream. The client receives this final chunk and concludes the streaming process.

During the streaming process, the client and server maintain an open connection (by setting the connection to "keep-alive" for this header to have this effect), allowing for the continuous flow of data chunks. The client can process and display the received chunks in real time, providing a responsive and interactive user experience.